



DAY 3



Computer Vision Preprocessing with OpenCV

Garbage Classification — From Raw Images to Model-Ready Input

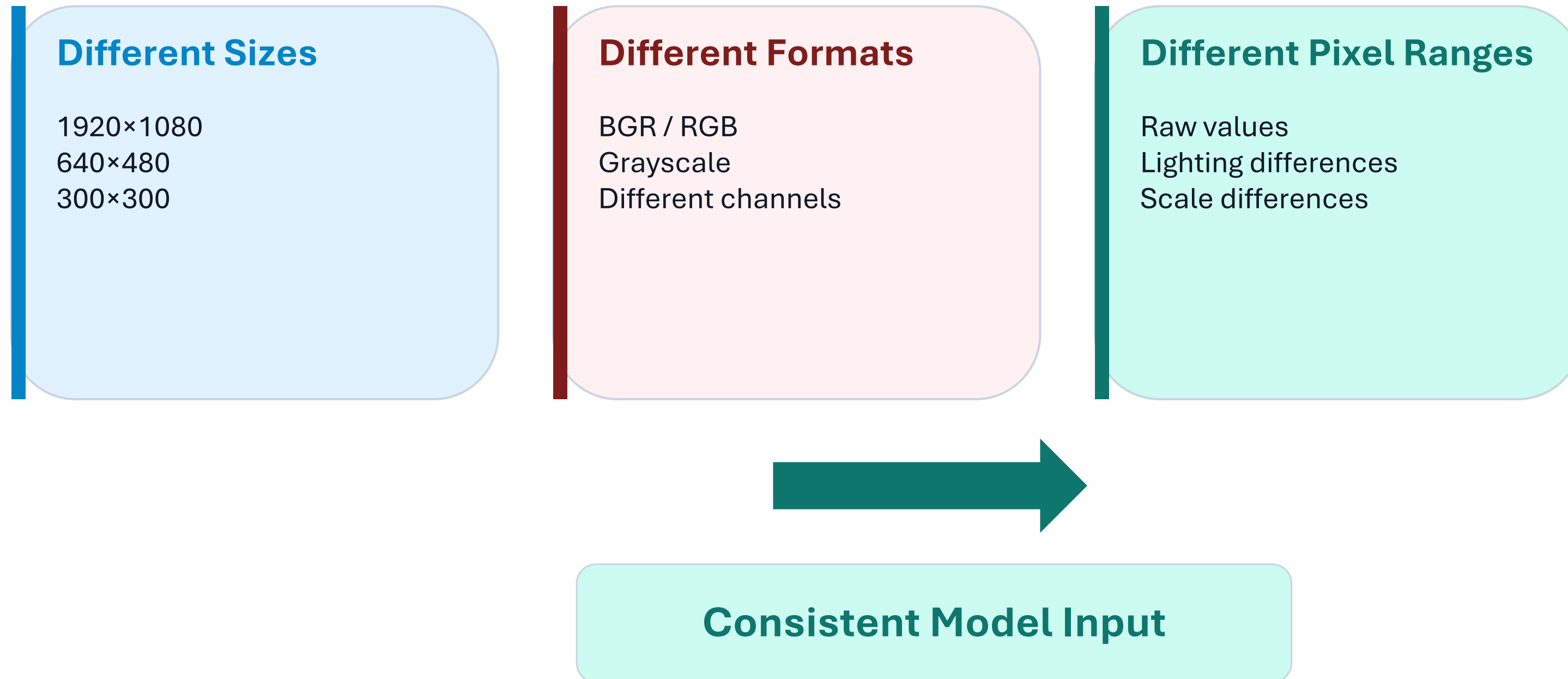
Week 8 • AI & ML Internship



Prepared By Sadeel Ardah

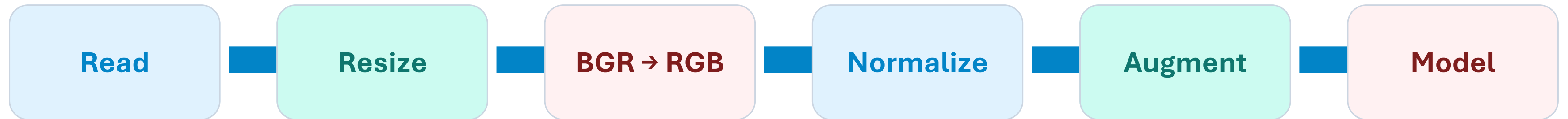
Why Images Need Preprocessing

Raw images are not automatically consistent



The Day 3 OpenCV Pipeline

From raw image to model-ready array

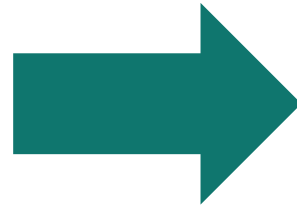


Goal: standardize the input without losing useful visual information.

Every image should reach the same spatial size

Before

Image A → 512×384
Image B → 300×300
Image C → 640×480



After cv2.resize()

All images
→ 224 × 224 × 3

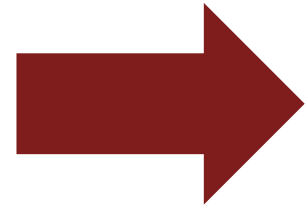
cv2.resize(image, (224, 224))

One of the most common OpenCV mistakes

OpenCV

`cv2.imread()`
reads color channels as:

B G R



Display / Deep Learning

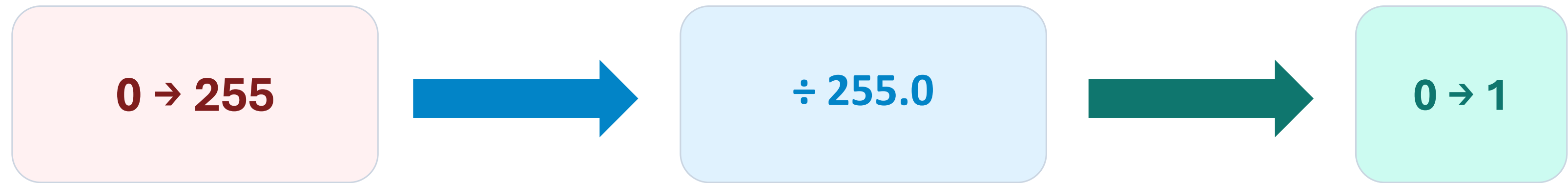
Usually expects:

R G B

`cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`

Normalization

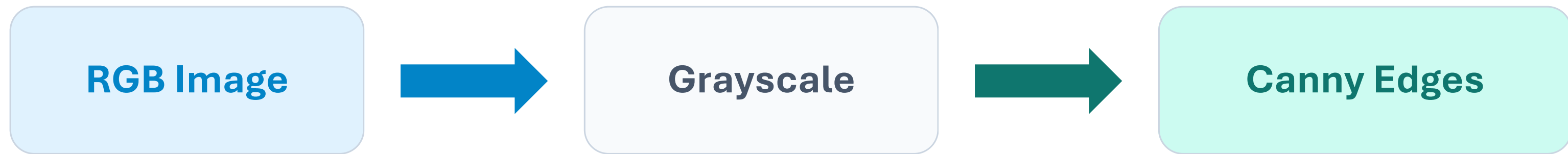
Scale pixel values into a model-friendly range



Generic normalization

```
image = image.astype('float32') / 255.0
```

Classical OpenCV can highlight strong boundaries



`edges = cv2.Canny(gray, 100, 200)`

Useful for understanding traditional computer vision; CNNs learn many edge-like features automatically.

Reusable Preprocessing Function

Combine the core operations into one pipeline

```
def preprocess_image(path):  
    image = cv2.imread(str(path))  
    image = cv2.resize(image, (224, 224))  
    image = cv2.cvtColor(  
        image,  
        cv2.COLOR_BGR2RGB  
    )  
    image = image.astype("float32")  
    image = image / 255.0  
    return image
```

Output Shape

$224 \times 224 \times 3$

Output Type

float32

Pixel Range

$0 \rightarrow 1$

Data Augmentation

One image can become many realistic training variations



Why?

Less memorization
Better generalization

Rule

Augment training images only — not validation or test.

Transfer Learning Preprocessing

The preprocessing must match the pre-trained model

Generic Pipeline

`image / 255.0`

Typical range:
 $0 \rightarrow 1$

MobileNetV2

`preprocess_input()`

Typical range:
 $\approx -1 \rightarrow 1$

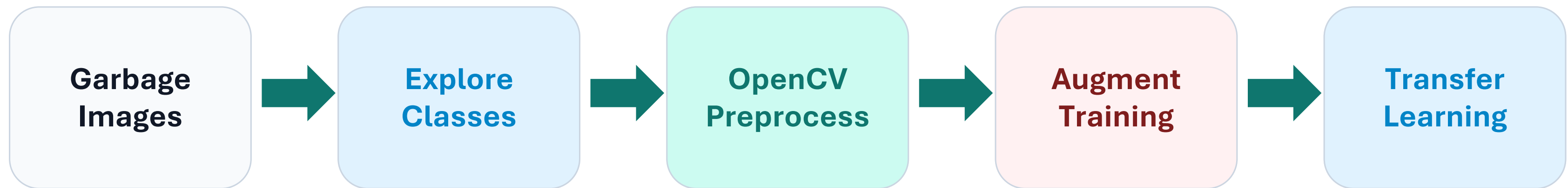
Key Warning

Do not normalize twice.
Use the preprocessing rule
expected by the selected model.

Correct preprocessing is part of the model.

Garbage Classification Application

A real-world Day 3 workflow



The goal is not just to display images — it is to build a reliable input pipeline for classification.

What I Learned Today

- 1 Read and resize images with OpenCV
- 2 Fix the BGR $\leftrightarrow$ RGB color-order issue
- 3 Normalize pixel values
- 4 Apply Canny edge detection
- 5 Build a reusable preprocessing function
- 6 Create data augmentation
- 7 Match preprocessing to transfer learning

KEY TAKEAWAY

**Consistent input +
correct
preprocessing = a
stronger computer
vision pipeline.**